



CASE Statement in SQL

A **CASE statement** in SQL is a conditional expression used to apply **if-else logic inside queries**. It allows you to evaluate conditions and return different values depending on whether those conditions are met.

It is commonly used for:

- classification of data
- creating labels
- conditional calculations
- business rules (salary, bonuses, grading systems)
- data transformation in reports

How CASE Works (Logic Flow)

A CASE statement works like:

1. Evaluate condition in WHEN
2. If TRUE → return value in THEN
3. If no conditions match → return ELSE (if provided)
4. End with END

Short Notes on Each Script Section

1. Database Selection

```
USE techsphere_solutions;
```

What it does:

- Selects the working database
- Ensures all queries run in TechSphere schema

2. View Employee Records

```
SELECT *  
FROM employees;
```

What it does:

- Displays all employee data
- Used as baseline data for CASE logic



3. Simple CASE (Basic Classification)

```
SELECT
    first_name,
    last_name,
    TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) AS age,
    CASE
        WHEN TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) <= 30
        THEN 'Young Professional'
    END AS age_category
FROM employees;
```

Query Purpose:

Classify employees based on age

What happens:

- Calculates employee age
- Assigns label only if age ≤ 30
- Others return NULL

4. Multi-Condition CASE (Age Classification)

```
SELECT
    first_name,
    last_name,
    TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) AS age,
    CASE
        WHEN TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) <= 30
        THEN 'Young Professional'
        WHEN TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) BETWEEN 31
        AND 45
        THEN 'Experienced Professional'
        WHEN TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE()) > 45
        THEN 'Senior Professional'
    END AS age_category
FROM employees;
```

What it does:

- Categorizes employees into 3 age groups
- Helps HR segment workforce by experience level



5. CASE with Salary Adjustment Logic

```
SELECT
  role_title,
  base_salary,
  CASE
    WHEN base_salary >= 120000
      THEN base_salary + (base_salary * 0.10)
    WHEN base_salary < 120000
      THEN base_salary + (base_salary * 0.05)
  END AS revised_salary
FROM job_roles;
```

What it does:

- Applies salary increment rules:
 - High salaries → 10% increase
 - Lower salaries → 5% increase
- Simulates payroll policy logic



5. CASE with Bonus Calculation (Multi-Table)

```
SELECT
    e.first_name,
    e.last_name,
    d.department_name,
    jr.base_salary,

    CASE
        WHEN jr.base_salary >= 120000
            THEN jr.base_salary + (jr.base_salary * 0.10)

        WHEN jr.base_salary < 120000
            THEN jr.base_salary + (jr.base_salary * 0.05)
    END AS revised_salary,

    CASE
        WHEN d.department_name = 'AI Research'
            THEN jr.base_salary * 0.15

        WHEN d.department_name = 'Cybersecurity'
            THEN jr.base_salary * 0.10

        ELSE 0
    END AS bonus

FROM employees e
JOIN employee_positions ep
    ON e.employee_id = ep.employee_id
JOIN job_roles jr
    ON ep.role_id = jr.role_id
JOIN departments d
    ON e.department_id = d.department_id;
```

**What it does:**

- Combines 3 tables (employees, roles, departments)
- Calculates:
 - revised salary (increment rules)
 - department-based bonus
- Used for payroll and HR analytics

7. Salary Grade Classification

```
SELECT
    role_title,
    base_salary,

    CASE
        WHEN base_salary >= 150000
            THEN 'Executive Level'

        WHEN base_salary BETWEEN 100000 AND 149999
            THEN 'Senior Level'

        WHEN base_salary BETWEEN 70000 AND 99999
            THEN 'Mid Level'

        ELSE 'Entry Level'
    END AS salary_grade

FROM job_roles;
```

What it does:

- Categorizes job roles into salary tiers
- Useful for HR hierarchy and compensation planning



8. Employee Cost Classification (Business Analytics)

```
SELECT
    CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
    d.department_name,
    jr.role_title,
    jr.base_salary,

    CASE
        WHEN jr.base_salary >= 140000
            THEN 'High Cost Employee'

        WHEN jr.base_salary BETWEEN 90000 AND 139999
            THEN 'Moderate Cost Employee'

        ELSE 'Low Cost Employee'
    END AS employee_cost_category

FROM employees e
JOIN employee_positions ep
    ON e.employee_id = ep.employee_id
JOIN job_roles jr
    ON ep.role_id = jr.role_id
JOIN departments d
    ON e.department_id = d.department_id;
```

What it does:

- Builds full employee profile
- Classifies employees based on cost to company
- Useful for budgeting and workforce planning

Key Concept Summary

- **CASE** = conditional logic inside SQL queries
- Used for:
 - Classification
 - business rules
 - computed fields
 - reporting logic

**Final Insight**

CASE statements turn raw database values into meaningful business intelligence by applying conditional logic directly inside SQL queries.